

Introduction Pratique à la Blockchain

Modules 1–4 : des fondements au réseau P2P

Abdoullah Ichou

EPSI — Auxerre

19 mai 2026

Progression pédagogique

- ➊ **Module 1 (1h30)** — Simulation « à la main » :
Bloc, chaînage, hachage, immuabilité
- ➋ **Module 2 (1h30)** — Proof of Work en Python :
Nonce, difficulté, sécurité par le calcul
- ➌ **Module 3 (1h)** — Cryptographie appliquée :
Clés ECDSA, signature, portefeuille
- ➍ **Module 4 (1h30)** — Blockchain P2P avec Flask :
Décentralisation, propagation, consensus

Objectif global

Construire pas à pas une blockchain fonctionnelle, du concept mathématique au réseau distribué.

Structure d'un bloc :

- **Index** — position dans la chaîne
- **Timestamp** — horodatage
- **Données** — transactions
- **Previous_Hash** — maillon
- **Hash** — empreinte propre

Bloc #1

```
index=1  
prev_hash=a3f...  
data="Alice->Bob"  
hash=7b2...
```

Module 1 — Hachage simplifié

Règle du TP (sans ordinateur)

Additionner la valeur numérique de chaque caractère :

A=1, B=2, ..., Z=26 chiffres = leur valeur espaces = 0

Puis prendre le **reste modulo 1000**.

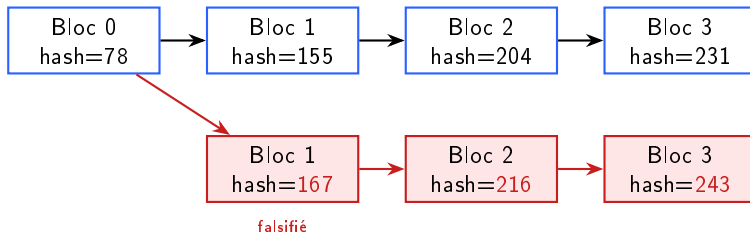
Exemple

$$\text{"GENESIS"} = G(7) + E(5) + N(14) + E(5) + S(19) + I(9) + S(19) = 78$$

Propriétés observées

- **Déterminisme** : même entrée $\Rightarrow$ même hash
- **Effet avalanche** : 1 caractère différent $\Rightarrow$ hash totalement différent

Scénario : un attaquant modifie le Bloc 1



Conclusion

Modifier un bloc oblige à recalculer **tous les blocs suivants**. Sur Bitcoin (avec PoW), cela représente des années de calcul.

Problème : deux blocs minés simultanément (fork)

- Le Groupe A mine le Bloc 4 avec "Alice -> Eve : 7"
- Le Groupe B mine le Bloc 4 avec "Alice -> Frank : 7"
- $\Rightarrow$ Deux chaînes valides coexistent temporairement

Règle de la chaîne la plus longue

Les nœuds adoptent toujours la chaîne représentant le plus de travail de calcul accumulé.

$\Rightarrow$ Le premier à miner le Bloc 5 **gagne**. L'autre bloc devient un **orphan block**.

Implication de sécurité

Un attaquant doit contrôler $> 51\%$ du hash rate pour réécrire l'historique (**attaque des 51%**).

Définition

La **Preuve de Travail** est un mécanisme qui impose qu'un bloc ne soit accepté que si son hash vérifie une **condition de difficulté** : commencer par d zéros hexadécimaux.

Algorithme :

- 1 Choisir un **nonce** = 0
- 2 Calculer $\text{hash}(\text{data} + \text{nonce})$
- 3 Si hash ne commence pas par d zéros : $\text{nonce} += 1$, retour en 2
- 4 Sinon : le bloc est miné!

d	Tentatives moy.
1	16
2	256
3	4 096
4	65 536

Module 2 — Implémentation Python

```
def mine_block(block, difficulty=4):
    prefix = "0" * difficulty
    while not block.hash.startswith(prefix):
        block.nonce += 1
        block.hash = block.calculate_hash()
    return block

def calculate_hash(self):
    content = (str(self.index) + self.previous_hash
              + str(self.timestamp) + str(self.data)
              + str(self.nonce))
    return hashlib.sha256(content.encode()).hexdigest()
```

Point clé

Le nonce est la seule variable. Trouver un hash valide est **difficile** ($O(16^d)$ tentatives) mais le **vérifier** est trivial (un seul SHA-256). C'est cette **asymétrie** qui sécurise la blockchain.

Résultats expérimentaux (Python, machine moderne)

Difficulté	Nonce moyen	Temps (s)	Facteur
1	~ 8	~ 0,0001	×1
2	~ 150	~ 0,001	×16
3	~ 2 500	~ 0,01	×16
4	~ 40 000	~ 0,15	×16
5	~ 600 000	~ 2,5	×16

Lien avec Bitcoin

Bitcoin ajuste la difficulté toutes les **2016 blocs** pour maintenir un temps de minage moyen de **10 minutes**, quelle que soit la puissance de calcul du réseau.

Pourquoi la PoW protège contre la falsification

Modifier un bloc ancien (ex : Bloc 1 sur une chaîne de 500 000 blocs) impose :

- 1 Recalculer le PoW du Bloc 1 modifié
- 2 Recalculer le PoW des 499 999 blocs suivants
- 3 Tout cela **plus vite** que le réseau honnête continue de miner

Validation : 3 vérifications

- 1 `current.hash == current.calculate_hash()` (données intègres)
- 2 `current.previous_hash == previous.hash` (chaînage)
- 3 `current.hash.startswith("0" * d)` (PoW valide)

Problème : comment prouver qu'Alice a autorisé une transaction, sans banque ni notaire ?

Solution : Cryptographie à courbe elliptique (ECC)

- **Clé privée** k — 256 bits aléatoires, secret absolu
- **Clé publique** $K = k \cdot G$ — dérivée, partageable
- **Sens unique** — K depuis k est facile ;
 k depuis K est impossible en pratique

Courbe SECP256k1

Utilisée par Bitcoin et Ethereum.

Clé privée : 32 bytes

Clé publique : 64 bytes

Signature : 64 bytes (paire r, s)

Protocole de signature

- 1 Alice calcule $h = \text{SHA256}(m)$ où m = la transaction
- 2 Elle génère $\sigma = \text{ECDSA_sign}(h, k_{\text{Alice}})$ avec sa clé **privée**
- 3 Elle diffuse (m, σ) sur le réseau
- 4 Tout le monde vérifie : $\text{ECDSA_verify}(\sigma, h, K_{\text{Alice}}) = \top$

Garanties

- **Authenticité** : seule Alice peut produire σ
- **Intégrité** : si m est modifié, σ devient invalide
- **Non-répudiation** : Alice ne peut pas nier avoir signé

Module 3 — Code clé en Python

```
from ecdsa import SigningKey, SECP256k1, BadSignatureError

# Generer une paire de clés
private_key = SigningKey.generate(curve=SECP256k1)
public_key = private_key.get_verifying_key()

# Signer une transaction
tx = "Alice -> Bob : 5 BTC"
sig = private_key.sign(tx.encode())

# Verifier
try:
    public_key.verify(sig, tx.encode())
    print("Signature valide")
except BadSignatureError:
    print("INVALIDE")
```

Dérivation d'une adresse Bitcoin (simplifiée)

$$\text{adresse} = \text{SHA256}(\text{SHA256}(\text{clé publique}))[:40]$$

En réalité, Bitcoin applique aussi RIPEMD-160 et Base58Check :

$$\text{adresse} = \text{Base58Check}(\text{RIPEMD160}(\text{SHA256}(K)))$$

Règle d'or

- La clé publique est votre **identité** (partagez-la)
- La clé privée est votre **mot de passe** (ne la partagez jamais)
- En Bitcoin, **pas de « mot de passe oublié »** : une clé perdue = fonds perdus

Chaque nœud expose une API REST

Route	Rôle
GET /chain	Retourner la blockchain complète
POST /transactions/new	Ajouter une transaction au mempool
GET /mine	Miner un nouveau bloc
POST /nodes/register	Enregistrer des pairs
GET /nodes/resolve	Exécuter le consensus
POST /blocks/receive	Recevoir un bloc d'un pair

Lancement

```
python tp4_node_starter.py -port 5000  
python tp4_node_starter.py -port 5001
```

Module 4 — Minage et propagation

```
@app.route('/mine', methods=['GET'])
def mine():
    if not mempool:
        return jsonify({'error': 'Mempool vide'}), 400
    last_hash = blockchain[-1]['hash']
    new_block = create_block(mempool.copy(), last_hash)
    mempool.clear()
    blockchain.append(new_block)
    broadcast_block(new_block) # propagation P2P
    return jsonify({'hash': new_block['hash']}), 200

def broadcast_block(block):
    for peer in peers:
        requests.post(f"{peer}/blocks/receive", json=block)
```

Principe

Dès qu'un bloc est miné, il est immédiatement propagé à tous les pairs enregistrés.

Règle de la chaîne la plus longue (Nakamoto)

- 1 Interroger chaque pair : `GET /chain`
- 2 Valider la chaîne reçue (chaînage + PoW + intégrité)
- 3 Si une chaîne valide **plus longue** est trouvée : l'adopter

Simulation d'un fork

- 1 Nœud A et Nœud B minent simultanément le Bloc 4
- 2 Deux chaînes valides coexistent
- 3 Le premier à miner le Bloc 5 “gagne”
- 4 `GET /nodes/resolve` sur l'autre nœud $\Rightarrow$ synchronisation

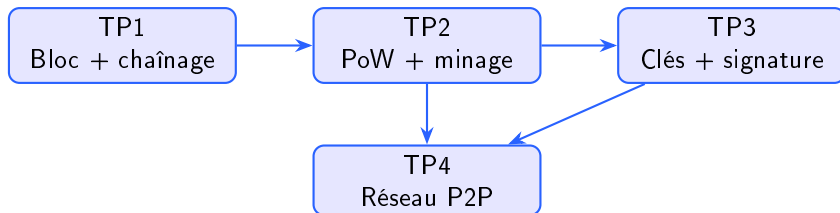
Forces

- **Sans autorité centrale** : le consensus émerge des interactions
- **Tolérance aux pannes** : un nœud hors ligne n'arrête pas le réseau
- **Transparence** : tout le monde peut vérifier la chaîne

Limites

- **Finalité probabiliste** : un bloc n'est « définitif » qu'après plusieurs confirmations
- **Attaque 51%** : un acteur majoritaire peut réécrire l'historique
- **Performance** : bien plus lent qu'une BDD centralisée

Synthèse — Ce que vous avez construit



Les 5 piliers démontrés en TP

- 1 **Immuabilité** — effet domino du hachage chaîné
- 2 **Sécurité** — coût énergétique du Proof of Work
- 3 **Identité** — cryptographie asymétrique ECDSA
- 4 **Décentralisation** — consensus P2P sans autorité
- 5 **Finalité** — règle de la chaîne la plus longue